



Lab #5 – Flags & Arithmetic

Objective

- Review Addition Instruction and learn Subtraction Instructions
- Understand the Status Register flags

Flags

In x86 architecture, the EFLAGS register contains a series of individual bits that represent the state of the processor after the execution of an instruction, often used for decision-making in programs. Each bit in the EFLAGS register is a flag that indicates different conditions and states.

The flags in the EFLAGS register are set for 8-bit, 16-bit, and 32-bit operations. The flags' behavior is consistent across these different operand sizes as explained below:

- 8-bit Operations: When performing operations on 8-bit registers (like AL or BL), the relevant flags (such as Zero Flag, Sign Flag, Overflow Flag, etc.) are set based on the result of that 8-bit operation.
- 16-bit Operations: Similarly, operations on 16-bit registers (like AX or BX) affect the flags based on the outcome of the 16-bit result. This was the default behavior for the original 8086 processor.
- 32-bit Operations: When operating on 32-bit registers (like EAX or EBX), the flags are updated based on the result of the 32-bit operation. In this mode, the carry, overflow, zero, sign, parity, and auxiliary flags are still set in response to the outcome, but the flags now consider the entire 32-bit result.

The following is a list of the flags that are affected by arithmetic instructions:

Flag		Meaning
Carry Flag	CF	Set to 1 when there is an unsigned overflow. For example, when you add bytes $255 + 1$ (result is not in range 0...255)
Zero Flag	ZF	Set to 1 when result is zero. For none zero result this flag is set to 0.
Sign Flag	SF	Set to 1 when result is negative. When result is positive it is set to 0.
Overflow Flag	OF	Set to 1 when there is a signed overflow. For example, when you add bytes $100 + 50$ (result is not in range -128...127).

Addition and Subtraction Instructions

The addition and subtraction functions are a subset of the arithmetic instructions which provide the ability to add or subtract data either between its internal registers or between an internal register and a storage location in memory. The addition and subtraction functions include:

Instruction	Description	Example	Operands
ADD	Adds the source operand to the destination	ADD EAX, EBX	reg, reg / reg, mem / mem, reg

	operand. Affects flags: OF, SF, ZF, CF.		/ reg, imm
INC	Increments the destination operand by 1. Does not affect CF. Affects flags: OF, SF, ZF.	INC ECX	reg / mem
DEC	Decrements the destination operand by 1. Does not affect CF. Affects flags: OF, SF, ZF.	DEC EDX	reg / mem
SUB	Subtracts the source operand from the destination operand. Affects flags: OF, SF, ZF, CF.	SUB EAX, EBX	reg, reg / reg, mem / mem, reg / reg, imm
NEG	Reverses the sign of the operand (two's complement negation). Affects flags: OF, SF, ZF, CF.	NEG EAX	reg / mem

Lab Work (Questions 1-5):

1. Initialize EBX = 80000000h and EDX = 80000000h, then write instructions to add EBX to EDX and store the value in EBX. Write down and explain any flags that are set after the operation.

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

val1 DWORD 80000000h

.code

main PROC

mov ebx, val1 ; CF = 0 SF = 0 ZF = 1 OF = 0

mov edx, val1 ; CF = 0 SF = 0 ZF = 1 OF = 0

add edx, ebx ; CF = 1 SF = 0 ZF = 1 OF = 1

mov ebx, edx ; CF = 1 SF = 0 ZF = 1 OF = 1

call dumpregs

exit

main ENDP

END main

Screenshot:

```
EAX=0019FFCC EBX=00000000 ECX=004010AA EDX=00000000
ESI=004010AA EDI=004010AA EBP=0019FF80 ESP=0019FF74
EIP=00403675 EFL=00000A47 CF=1 SF=0 ZF=1 OF=1 AF=0 PF=1
```

Explanation:

$edx + ebx = 1\ 0000\ 0000$, which is causing $CF = 1$, it exceeds the eax storage. $ZF = 0$ since $ebx = 00000000$. And it will get an $OF = 1$ since a $neg + neg = positive$, which triggers the overflow flag.

2. Execute the following two sets of instructions then explain which flags were affected in each case
 - i. `mov ax, 0FFFFh`
`add ax, 00001h`

Screenshot:

```
EAX=00190000 EBX=00237000 ECX=004010AA EDX=004010AA
ESI=004010AA EDI=004010AA EBP=0019FF80 ESP=0019FF74
EIP=0040366D EFL=00000257 CF=1 SF=0 ZF=1 OF=0 AF=1 PF=1
```

Explanation:

CF = 1 since $0FFFF + 000001 = 10000$ which is larger than the 16 bit register as. And in **ZF = 1**, since **ax = 0000** after adding the values.

- ii. `mov ax, 0FFFFh`
`inc ax`

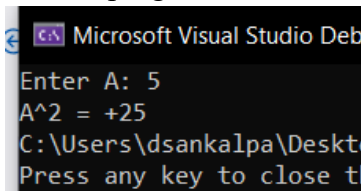
Screenshot:

```
EAX=00190000 EBX=0039F000 ECX=004010AA EDX=004010AA
ESI=004010AA EDI=004010AA EBP=0019FF80 ESP=0019FF74
EIP=0040366B EFL=00000256 CF=0 SF=0 ZF=1 OF=0 AF=1 PF=1
```

Explanation

CF = 0 since the `inc/dec` does not affect the register. And **ZF = 1** since it exceeds **ax**.

3. Write a program to calculate the square of number *A* print the result.



```
Microsoft Visual Studio Deb
Enter A: 5
A^2 = +25
C:\Users\dsankalpa\Desktop
Press any key to close t
```

Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```
str1 BYTE "Enter A: ", 0
str2 BYTE "A^2 = ", 0
sum DWORD 0
```

.code

main PROC

```
mov edx, OFFSET str1
call WriteString
mov eax, ReadDec
call ReadDec
```

```

        mov edx, OFFSET str2
        call WriteString

        mov ecx, eax

again:
        add sum, eax

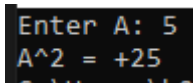
        loop again

        mov eax, sum
        call WriteInt

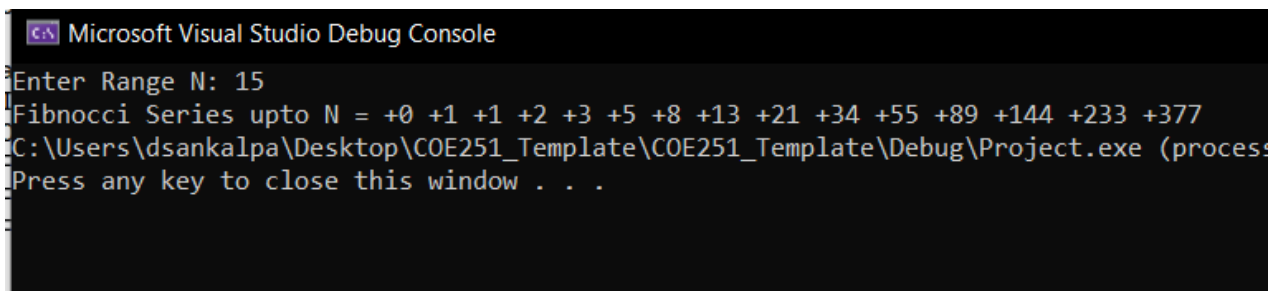
        exit
main ENDP
END main

```

Screenshot:



- Write a program to generate and display the Fibonacci series up to the N-th term. The user should be able to input the number of terms N. The Fibonacci series starts with 0 and 1, and each subsequent number is the sum of the previous two numbers in the series.



Solution:

TITLE Name/Purpose Here

INCLUDE Irvine32.inc

.data

```

str1 BYTE "Enter Range N: ", 0
str2 BYTE "Fibonacci Series upto N = ", 0
str3 BYTE " ", 0
fib0 DWORD 0
fib1 DWORD 1

```

.code

main PROC

```

        mov edx, OFFSET str1
        call WriteString

```

```

        call Readint
        mov ecx, eax

```

```

    mov edx, OFFSET str2
    call WriteString

again:
    mov ebx, fib1
    mov eax, fib0
    call WriteInt

    mov edx, OFFSET str3
    call WriteString

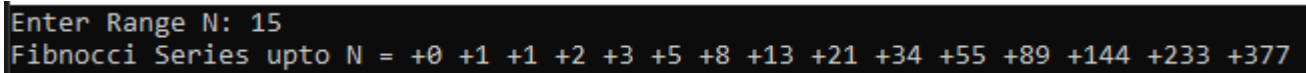
    add eax, ebx
    mov fib1, eax
    mov fib0, ebx

    loop again

    exit
main ENDP
END main

```

Screenshot:



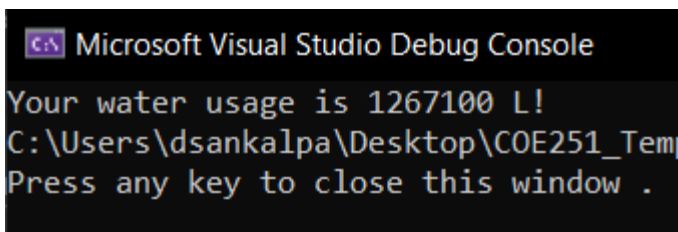
```

Enter Range N: 15
Fibonacci Series upto N = +0 +1 +1 +2 +3 +5 +8 +13 +21 +34 +55 +89 +144 +233 +377

```

- Write a program to calculate the total water usage of a facility over five days. The water usage for each of the five days (in liters) is as follows (all values are decimal): 205050, 312400, 150300, 423100, and 176250.

Hint: Remember that each DWORD occupies four bytes, so you'll need to adjust your offsets in the array accordingly.



```

Microsoft Visual Studio Debug Console
Your water usage is 1267100 L!
C:\Users\dsankalpa\Desktop\C0E251_Tem
Press any key to close this window .

```

Solution:

TITLE Name/Purpose Here

```
INCLUDE Irvine32.inc
```

```
.data
```

```

str1 BYTE "Your water usage is ", 0
str2 BYTE " L!", 0
val1 DWORD 205050, 312400, 150300, 423100, 176250
sum DWORD 0

```

```
.code
```

```
main PROC
```

```

    mov edx, OFFSET str1
    call WriteString

```

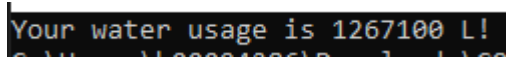
```
    mov eax, 0
    add eax, val1
    add eax, val1+4
    add eax, val1+8
    add eax, val1+12
    add eax, val1+16

    call WriteDec

    mov edx, OFFSET str2
    call WriteString

    exit
main ENDP
END main
```

Screenshot:



```
Your water usage is 1267100 L!

```